PLAN DE PRUEBAS

TuHospedaje

Plan y Reporte de Pruebas de Software — Sprint 2

TuHospedaje — Seguridad y Organización

TC-12 a TC-20
CASOS

49 tests sin fallos
AUTOMATIZADAS

Equipo de Desarrollo · Mayo 2026

Foco del Incremento: Seguridad (JWT), Roles de Usuario, Categorización y Asignación de Características
Enfoque de Testing: Pruebas de API (Postman), Verificación de UI Manual y Pruebas Automatizadas (JUnit 5 + MockMvc)

1. Matriz Detallada de Casos de Prueba (Test Cases)

TC-12: Módulo de Categorías (CRUD e Integridad Referencial)

- **Historias de Usuario Asociadas:** US #12 (Categorizar productos) y US #21 (Agregar categoría).
- **Precondiciones:** Servidor de backend e instancia de MariaDB activos; Token JWT de Administrador provisto en la cabecera HTTP.
- **Tipos de Verificación:** API Rest, Interfaz de Usuario (UI) y Test de Integración Automatizado.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`POST /api/categories` con Token Admin y JSON válido	HTTP 201 Created. JSON con `id`, `name` y `description`	✔ Pasa
2	`POST /api/categories` enviando un nombre ya existente en BD	HTTP 400 Bad Request. Mensaje de validación	✔ Pasa
3	`GET /api/categories` (Acceso Anónimo / Público)	HTTP 200 OK. Array de objetos JSON con el catálogo	✔ Pasa
4	`GET /api/categories/{id}` con clave primaria existente	HTTP 200 OK. Estructura de datos completa de la categoría	✔ Pasa
5	`GET /api/categories/{id}` con clave primaria inexistente	HTTP 404 Not Found	✔ Pasa
6	`PUT /api/categories/{id}` con Token Admin y payload modificado	HTTP 200 OK. Retorna el DTO actualizado	✔ Pasa
7	`DELETE /api/categories/{id}` sobre categoría sin alojamientos asociados	HTTP 204 No Content	✔ Pasa
8	`DELETE /api/categories/{id}` sobre categoría con alojamientos asociados	HTTP 400 Bad Request. Bloqueo por integridad referencial	✔ Pasa
9	`POST /api/categories` de forma anónima (Sin cabecera Bearer)	HTTP 401 Unauthorized. Acceso denegado por el filtro de seguridad	✔ Pasa
10	`POST /api/categories` con token de usuario `ROLE_USER`	HTTP 403 Forbidden. Token válido pero privilegios insuficientes	✔ Pasa
11	UI: Admin → Categorías → "Crear"	Apertura de modal; inserción y refresco asíncrono de tabla	✔ Pasa
12	UI: Admin → Categorías → "Editar"	Modal con datos precargados	✔ Pasa
13	UI: Admin → Categorías → "Eliminar"	`ConfirmDialog`; al confirmar, remueve la fila	✔ Pasa

TC-13: Flujo de Registro de Cuentas de Usuario

- **Historia de Usuario Asociada:** US #13 (Registrar usuario).
- **Precondiciones:** Backend en ejecución; el email no debe existir en BD.
- **Tipos de Verificación:** API Rest, UI Manual y Pruebas Automatizadas (MockMvc).

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`POST /api/auth/register` con payload válido completo	HTTP 201 Created. Retorna JWT con claims	✔ Pasa
2	`POST /api/auth/register` con email duplicado	HTTP 400 Bad Request. "El email ya está registrado"	✔ Pasa
3	`POST /api/auth/register` omitiendo `firstName`	HTTP 400 Bad Request. Error de validación	✔ Pasa
4	`POST /api/auth/register` con password < 6 caracteres	HTTP 400 Bad Request. Longitud insuficiente	✔ Pasa
5	UI: Navegar a `/register`	Formulario estructurado de registro visible	✔ Pasa
6	UI: Completar formulario con datos válidos y enviar	Redirección al Home; mutación de UI con nuevo Avatar	✔ Pasa
7	UI: Feedback reactivo de contraseña	Indicadores ✘/✔ mutan en tiempo real	✔ Pasa

TC-14: Autenticación, Emisión de JWT y Control de Rutas

- **Historia de Usuario Asociada:** US #14 (Identificar usuario).
- **Precondiciones:** Cuenta de usuario previamente registrada.
- **Tipos de Verificación:** API Rest, UI Manual y Pruebas Automatizadas.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`POST /api/auth/login` con credenciales correctas	HTTP 200 OK. Retorna JWT con expiración 8h	✔ Pasa
2	`POST /api/auth/login` con email inexistente	HTTP 401 Unauthorized. "Credenciales inválidas"	✔ Pasa
3	`POST /api/auth/login` con password incorrecta	HTTP 401 Unauthorized	✔ Pasa
4	UI: Login con cuenta `ROLE_ADMIN`	Header renderiza avatar + nombre "Admin"	✔ Pasa
5	UI: Clic en avatar de Administrador	Redirección declarativa a `/admin`	✔ Pasa
6	UI: Login con cuenta `ROLE_USER`	Header muestra avatar; botones de admin ocultos	✔ Pasa

TC-15: Finalización de Sesión (Logout)

- **Historia de Usuario Asociada:** US #15 (Cerrar sesión).
- **Precondiciones:** Sesión de usuario activa (JWT en localStorage).
- **Tipos de Verificación:** UI Manual.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	Estando logueado, pulsar "Cerrar sesión"	Eliminación del token en `localStorage`; UI vuelve a modo anónimo	✔ Verificado
2	Recarga forzada del navegador (F5)	Estado anónimo persiste sin fugas de memoria	✔ Verificado
3	Navegación directa a `/admin` sin sesión	Redirección preventiva por protección de ruta	✔ Verificado

TC-16: Panel de Administración de Cuentas y Roles

- **Historia de Usuario Asociada:** US #16 (Identificar administrador).
- **Precondiciones:** Token JWT de Administrador activo.
- **Tipos de Verificación:** API Rest e UI Manual.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`GET /api/users` con token ADMIN válido	HTTP 200 OK. Lista completa de usuarios	✔ Pasa
2	`GET /api/users` sin token o con rol común	HTTP 401 o 403 según contexto	✔ Pasa
3	`PUT /api/users/{id}/role` con `{"role":"ADMIN"}`	HTTP 200 OK. Rol actualizado	✔ Pasa
4	UI: Fila del administrador en sesión	Botón de cambio de rol inhabilitado	✔ Verificado
5	UI: Admin → Usuarios	Tabla con ID, Nombre, Email, Rol, Acciones	✔ Verificado
6	UI: Cambiar rol de usuario a ADMIN	Etiqueta visual muta; acceso habilitado	✔ Verificado

TC-17: Módulo Maestro de Características (Amenities)

- **Historia de Usuario Asociada:** US #17 (Administrar características).
- **Precondiciones:** Token de administrador activo.
- **Tipos de Verificación:** API Rest e UI Manual.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`POST /api/features` con token ADMIN y body (name, icon)	HTTP 201 Created	✔ Pasa
2	`GET /api/features` sin autenticación (Público)	HTTP 200 OK. Listado completo con íconos	✔ Pasa
3	`PUT /api/features/{id}` modificando nombre	HTTP 200 OK	✔ Pasa
4	`DELETE /api/features/{id}` sin asignaciones activas	HTTP 204 No Content	✔ Pasa
5	UI: Admin → Características → "Crear Nueva"	Modal captura datos; tabla se actualiza	✔ Verificado
6	UI: Modal de creación/edición de Alojamiento	Características se renderizan como checkboxes	✔ Verificado

TC-18: Visualización Dinámica de Equipamiento en Ficha de Producto

- **Historia de Usuario Asociada:** US #18 (Visualizar características).
- **Precondiciones:** Alojamiento con características asignadas en BD.
- **Tipos de Verificación:** Estructura JSON y UI Responsiva.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`GET /api/lodgings/{id}` con features asignadas	JSON incluye colección `features` (`id`, `name`, `icon`)	✔ Pasa
2	UI: Detalle de alojamiento (`/lodging/:id`)	Bloque "¿Qué ofrece este lugar?" visible	✔ Verificado
3	UI: Correspondencia ícono + nombre	Cada feature expone su icono y etiqueta	✔ Verificado
4	UI: Reducir ventana del navegador	Grilla se reordena por Media Query	✔ Verificado

Cobertura automatizada (frontend) — agregada en auditoría posterior: los pasos 2 y 3 no tenían assertion automatizada hasta entonces (solo verificación manual). `ProductDetail.test.jsx` — `describe('ProductDetail - Features detail', ...)` ahora verifica que las features reales del alojamiento se rendericen por nombre e ícono, y que la sección no aparezca cuando no hay features (commit `2bced9d`).

TC-19: Motor de Notificaciones Asíncronas por Correo Electrónico

- **Historia de Usuario Asociada:** US #19 (Email confirmación registro).
- **Precondiciones:** Backend con perfil Mailtrap activo.
- **Tipos de Verificación:** Trazabilidad de Logs y Sandbox externo.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	Registrar usuario desde la UI	Log: "Email de bienvenida enviado a..."	✔ Verificado
2	Inspeccionar Mailtrap inbox	Correo recibido con nombre, email y link de login	✔ Verificado
3	Simular fallo SMTP	Backend captura error en logs; registro en UI finaliza con éxito	✔ Verificado

Cobertura automatizada (backend) — corrección de auditoría posterior: el paso 3 no estaba realmente garantizado hasta la corrección de transaccionalidad de este pase: `AuthServiceImpl.register()` invocaba el envío del email dentro de la misma transacción que persiste el usuario, y `SmtpEmailServiceImpl.send()` no atrapaba `MailException` (la excepción real que lanza un fallo de SMTP) — un fallo SMTP real podía revertir el registro. Corregido: el envío ahora se difiere a `TransactionSynchronization#afterCommit`, con la excepción atrapada y logueada, nunca propagada. Test: `AuthServiceImplTest.shouldSendWelcomeEmailOnRegister` verifica que el email se dispara al registrar. Commits `2bced9d` y `091df56`.

TC-20: Motor de Filtrado por Categorías en el Home

- **Historia de Usuario Asociada:** US #20 (Crear sección de categorías).
- **Precondiciones:** Registros en tablas `LODGING` y `CATEGORY`.
- **Tipos de Verificación:** API REST y comportamiento UI.

PASO	ACCIÓN / ESTÍMULO DE PRUEBA	RESULTADO ESPERADO (CRITERIO DE ACEPTACIÓN)	ESTADO
1	`GET /api/lodgings?category={id}` con categoría poblada	HTTP 200 OK. Subconjunto filtrado de alojamientos	✔ Pasa
2	`GET /api/lodgings?category={id}` sin alojamientos	HTTP 200 OK. Arreglo vacío `[]`	✔ Pasa
3	UI: Cargar Home	Categorías visibles como chips interactivos	✔ Verificado
4	UI: Clic en categoría	Grilla se limpia y renderiza alojamientos filtrados	✔ Verificado
5	UI: Clic en "Mostrar todos"	Filtro destruido; UI vuelve a recomendaciones aleatorias	✔ Verificado

2. Resumen General de Ejecución del Sprint 2

CÓDIGO	MÓDULO / HISTORIA DE USUARIO	NATURALEZA DE LA PRUEBA	CONDICIÓN FINAL

TC-12	US #12 / #21 — CRUD de Categorías	API + UI + Integración JUnit	✔ Pasa
TC-13	US #13 — Registro de Usuarios	API + UI + Integración JUnit	✔ Pasa
TC-14	US #14 — Autenticación y JWT	API + UI + Integración JUnit	✔ Pasa
TC-15	US #15 — Cierre de Sesión Cliente	Verificación Manual de Estado	✔ Verificado
TC-16	US #16 — Panel de Gestión de Roles	API REST + UI + Integración JUnit	✔ Pasa
TC-17	US #17 — CRUD Maestro de Características	API REST + UI + Integración JUnit	✔ Pasa
TC-18	US #18 — Render de Amenities en Ficha	UI Manual + Diseño Adaptativo	✔ Verificado
TC-19	US #19 — Triggers de Correo Saliente	Flujo Asíncrono + Mailtrap	✔ Verificado
TC-20	US #20 — Navegación por Filtros	API REST + Estado de React	✔ Verificado

3. Métricas Consolidadas de Aseguramiento de Calidad (QA)

- **Batería de Pruebas Automatizadas (Suite JUnit 5 + MockMvc):** 49 Casos de prueba ejecutados de forma exitosa (0 fallos detectados). Cobertura completa sobre CategoryController (9 tests), AuthController (5 tests), y los nuevos FeatureController (11 tests) y UserController (7 tests). Los servicios se cubren con 17 tests unitarios adicionales (Mockito).
- **Manejo de Pruebas de Regresión:** La inclusión sistemática de la anotación `@Transactional` en las suites automáticas garantizó que el estado de los datos de prueba se reiniciara tras cada estímulo, protegiendo la integridad de la base de datos de desarrollo.
- **Aislamiento Total con Testcontainers:** Los tests de integración `@SpringBootTest` se ejecutan contra una instancia efímera de MariaDB 10.11 levantada en Docker via Testcontainers (v1.21.4). Esto elimina la dependencia de la base de datos de desarrollo, previene colisiones con datos existentes y garantiza un estado limpio por cada ejecución. La integración con Spring Boot 3.5 se realiza mediante `@ServiceConnection`, sin necesidad de configuración manual de propiedades de conexión.
- **Estado del Incremento: APROBADO PARA DESPLIEGUE CONTINUO**. Las limitaciones funcionales remanentes han sido catalogadas como deuda técnica controlada, sin bloqueos críticos en los flujos principales del sistema.